

Fitness Center Song Request Web App

Planning Document: product structure, stack, data model, and MVP build order

1. Product Goal

Build a simple song request queue for the fitness center. Gym users scan a QR code, search for a song, and submit a request. Staff see requests in first-in, first-out order and can mark each request as completed or rejected.

2. Core User Flows

Role	What They Do	Main Result
Gym user	Scans QR code, searches for a song, chooses a suggestion, submits request.	Song request is saved as pending.
Staff	Opens dashboard, views queue, completes or rejects requests.	Queue stays organized and manageable.
App	Applies daily limit, stores request, orders requests by time.	Fair FIFO request system with basic spam control.

3. Pages and Features

User Request Page: /request

- Headline: "Ask us to play a song".
- Search box for song title or artist.
- Live recommendations while typing.
- Submit selected song.
- Show daily request limit, such as 3 songs per day.
- Success message after submission.

Staff Dashboard: /staff

- Shows pending requests in FIFO order.
- Displays song title, artist, time requested, and optional album image.
- Buttons: Complete and Reject.
- Auto-refreshes every 5 seconds for version 1.
- Can later support WebSockets for instant updates.

History/Admin Page: /history, future version

- Shows completed and rejected songs.
- Shows most requested songs.

- Helps staff see trends and repeated requests.

4. Recommended Stack

Layer	Recommendation	Reason
Frontend	React + TypeScript + Vite	Fast, clean, and matches your existing React experience.
Backend	FastAPI + Python	Simple API development and strong fit with your Python background.
Database	SQLite for local development; PostgreSQL for deployment	SQLite is easy at first; PostgreSQL is better for production.
Deployment	Vercel + Render + Supabase PostgreSQL	Common beginner-friendly deployment path for frontend, backend, and database.
Realtime updates	Start with polling every 5 seconds	Good enough for MVP and easier than WebSockets.

5. High-Level Architecture

Gym user scans QR code -> React request page -> Backend API -> Music search API -> Song suggestions -> User submits -> Database saves pending request -> Staff dashboard reads pending queue -> Staff completes or rejects request.

6. Backend API Routes

Route	Method	Purpose
/api/search-songs?q=...	GET	Return song suggestions from Spotify or another music API.
/api/requests	POST	Create a new song request after checking the daily limit.
/api/staff/queue	GET	Return pending requests ordered by requested_at ascending.
/api/staff/requests/:id/complete	PATCH	Mark a request as completed.
/api/staff/requests/:id/reject	PATCH	Mark a request as rejected with optional reason.

7. Data Model

Main table: song_requests

- id
- song_title
- artist_name
- album_name
- spotify_track_id
- album_image_url
- requested_at
- status: pending, completed, rejected

- completed_at
- rejected_at
- rejection_reason
- user_device_id
- user_ip_hash
- request_date

FIFO query: SELECT * FROM song_requests WHERE status = 'pending' ORDER BY requested_at ASC;

8. No-Login Request Limit

Because the app does not require user accounts, use a browser/device ID stored in localStorage plus a hashed IP address and date. For version 1, limit each device/browser to 3 requests per day. This is not perfect, but it is enough for a low-risk gym environment.

9. Music Search Options

Option	Pros	Cons	Recommendation
Spotify Web API	Best search quality and recognizable song metadata.	Needs developer setup and rate-limit handling. Development-mode search results are now more limited.	Best user experience.
MusicBrainz API	Open music database and free to use.	Must respect about one request per second; search may feel less polished.	Good open alternative.
Deezer API	Has music search and a developer API.	Requires accepting API terms; ecosystem may be less familiar.	Backup option.

Recommended for MVP: use Spotify for suggestions, but design the app so you can swap the music search provider later if needed.

10. MVP Build Order

1. Create database schema.
2. Build backend API for submitting requests.
3. Build backend API for staff queue.
4. Build basic user request page with manual song input.
5. Build staff dashboard with complete/reject actions.
6. Add Spotify search suggestions.
7. Add daily request limit.
8. Generate QR code and test on a phone.
9. Deploy frontend, backend, and database.

11. Suggested Folder Structure

```
song-request-app/  
  frontend/  
    src/  
      pages/  
        RequestPage.tsx  
        StaffDashboard.tsx  
      components/  
        SongSearchBox.tsx  
        SongSuggestionList.tsx  
        QueueItem.tsx  
      api/  
        client.ts  
      App.tsx  
  backend/  
    app/  
      main.py  
      routes/  
        songs.py  
        requests.py  
      models/  
        song_request.py  
      services/  
        spotify_service.py  
        request_limit_service.py  
      database.py  
      requirements.txt  
  README.md
```

12. Future Features

- Voting instead of duplicate requests.
- Explicit-song filtering or staff warning labels.
- Recently played warning.
- Analytics: most requested songs and peak request times.
- Different request pages for different gym zones.
- Admin PIN or simple staff password for dashboard access.
- WebSockets for instant dashboard updates.

13. Reference Notes

- Spotify Web API rate limits are based on calls in a rolling 30-second window:
<https://developer.spotify.com/documentation/web-api/concepts/rate-limits>
- Spotify February 2026 migration guide notes development-mode search endpoint limit changes:
<https://developer.spotify.com/documentation/web-api/tutorials/february-2026-migration-guide>
- MusicBrainz API asks clients to keep requests to about one request per second:
https://musicbrainz.org/doc/MusicBrainz_API
- Deezer developer API page: <https://developers.deezer.com/api>